

Physics-Informed Neural Networks versus Classical Least Squares for Estimating Dengue Transmission Parameters: A Case Study of Bulacan, Philippines (1994–2010)



Almoguez, Romar S.
Dela Cruz, Sean Arkin C.
Fajardo, Osyris Benedict P.
Hornales, Jian Kyle C.

Bachelor of Science in Mathematics major in Computer Science
College of Science, Bulacan State University, Malolos, Bulacan, Philippines

MCS 306 Final Course Output

Code and data: `https://github.com/SeanXxotick/MCS306FCO`

Abstract

Estimating how fast a disease spreads is central to outbreak response, yet the transmission and recovery rates of epidemic models cannot be measured directly and must be inferred from case data. This study compares two approaches to this inverse problem for the susceptible–infected–recovered (SIR) model: a classical solve-then-fit method that couples a fourth-order Runge–Kutta (RK4) solver with nonlinear least squares, and a physics-informed neural network (PINN) that learns the solution and the parameters simultaneously by penalizing violations of the differential equations. A step-size study confirms that the RK4 solver converges with observed order 3.99, matching the theoretical order of four. On synthetic outbreaks with 10% observation noise, the PINN recovered the transmission rate β and recovery rate γ with relative errors of 0.06% and 1.25%, comparable to the classical method (0.53% and 0.81%, averaged over 5 noise draws), but required about 118 times more computation time and was sensitive to the weight of its physics loss. Applied to monthly dengue counts from Bulacan (OpenDengue, 6 outbreak seasons between 1998 and 2010), the classical SIR fit reproduced seasonal curves with R^2 between 0.903 and 0.998 and effective reproduction numbers between 1.29 and 3.43. With only eight monthly observations per season, however, the PINN's learned parameters did not reproduce its own fitted curve (R^2 of the re-simulated curve: 0.65). The results suggest that for low-dimensional ODE models with sparse data, classical fitting remains the more reliable and economical tool, while PINNs need richer data or additional constraints to be reliable.

1 Introduction

Dengue is a mosquito-borne viral infection and one of the most widespread vector-borne diseases in the world, with an estimated 390 million infections per year (Bhatt et al., 2013). In the Philippines, dengue is endemic and peaks during the rainy season, placing recurring pressure on local health systems. Bulacan, a rapidly urbanizing province in Central Luzon (Region III), has experienced repeated outbreaks; reported monthly cases in the province rose above one thousand during the 2010 season (OpenDengue; Clarke et al., 2024).

Mathematical models help turn case counts into quantities that guide decisions. The SIR model of Kermack and McKendrick (1927) summarizes an outbreak with two rates: the transmission rate β and the recovery rate γ, whose ratio $R_0 = \beta/\gamma$ (the basic reproduction number) determines whether an outbreak grows. These rates are not observed directly; they must be estimated from data, which makes parameter estimation an inverse problem. The standard approach repeatedly solves the differential equations numerically and adjusts the parameters to minimize the misfit with data.

Physics-informed neural networks (PINNs; Raissi et al., 2019) offer an artificial-intelligence alternative: a neural network approximates the solution of the differential equations, and the equations themselves enter the training loss, so the network fits the data while obeying the physics. Unknown parameters such as β and γ are learned alongside the network weights. PINNs have attracted wide interest in scientific machine learning (Karniadakis et al., 2021), but they are also known to be difficult to train (Wang et al., 2021), and their advantage over classical methods for small epidemic models is not obvious.

This study therefore asks: for estimating SIR parameters from dengue data, how does a PINN compare with a classical RK4 plus least-squares method in accuracy, stability and computational cost? We (1) verify the numerical solver through a step-size convergence study; (2) compare both methods on synthetic outbreaks with known parameters and controlled noise; (3) examine the PINN's sensitivity to its physics-loss weight; and (4) apply both methods to real monthly dengue counts from Bulacan. The significance of the study is practical: it tells local modelers which tool is appropriate for the sparse, aggregated surveillance data typically available at the provincial level, and it identifies what a PINN-based approach would need in order to be useful.

2 Preliminaries

2.1 The SIR Epidemic Model

Let $S(t)$, $I(t)$ and $R(t)$ denote the fractions of a closed population that are susceptible, infected (and infectious) and recovered at time t (in days). The SIR model is the system of ordinary

differential equations (ODEs)

$$\frac{dS}{dt} = -\beta SI, \qquad \frac{dI}{dt} = \beta SI - \gamma I, \qquad \frac{dR}{dt} = \gamma I \tag{1}$$

with initial condition $(S,I,R)(0) = (1-I_0, I_0, 0)$. The term βSI models new infections under the mass-action assumption, and γI models recoveries, so $1/\gamma$ is the mean infectious period. Adding the three equations gives $d(S+I+R)/dt = 0$, so $S+I+R = 1$ for all t. Linearizing dI/dt near $S \approx 1$ gives $dI/dt \approx (\beta-\gamma)I$, so infections grow initially if and only if $R_0 = \beta/\gamma > 1$.

2.2 The Fourth-Order Runge–Kutta Method

System (1) has no closed-form solution, so it is solved numerically. Writing $\mathbf{y} = (S,I,R)$ and $d\mathbf{y}/dt = \mathbf{f}(\mathbf{y})$, the classical fourth-order Runge–Kutta (RK4) method advances the solution with a fixed step size h by

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{f}(\mathbf{y}_n), & \mathbf{k}_2 &= \mathbf{f}\big(\mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_1\big), \\ \mathbf{k}_3 &= \mathbf{f}\big(\mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_2\big), & \mathbf{k}_4 &= \mathbf{f}(\mathbf{y}_n + h\mathbf{k}_3), \end{aligned} \tag{2}$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right). \tag{3}$$

RK4 has local truncation error $O(h^5)$ and global error $O(h^4)$ for smooth $\mathbf{f}$ (Burden and Faires, 2011). If $e(h)$ is the global error with step h, the observed order of accuracy between two step sizes is

$$p = \frac{\log\big(e(h_1)/e(h_2)\big)}{\log(h_1/h_2)}, \tag{4}$$

which should approach 4 as h decreases.

2.3 Nonlinear Least Squares

Given observations y_i at times t_i and a model prediction $m(t_i;\theta)$ that depends on parameters θ, nonlinear least squares chooses θ to minimize

$$\Phi(\theta) = \tfrac{1}{2}\sum_i r_i(\theta)^2, \qquad r_i(\theta) = m(t_i;\theta) - y_i. \tag{5}$$

We use the trust-region reflective algorithm in SciPy's `least_squares` (Virtanen et al., 2020), a Gauss–Newton-type method that handles bound constraints (here $\beta, \gamma > 0$). Each evaluation of the residual requires a full RK4 solution of (1), and the Jacobian is approximated by finite differences, which costs one further solution per parameter at each iteration. Iteration stops when the relative changes in Φ, in θ, or the gradient norm fall below a tolerance.

2.4 Neural Networks and Automatic Differentiation

A fully connected neural network with L hidden layers maps an input x to an output through compositions $z_\ell = \sigma(W_\ell z_{\ell-1} + b_\ell)$, where W_ℓ and b_ℓ are trainable weights and biases and σ is a nonlinear activation (here tanh). Such networks can approximate any continuous function on a bounded domain arbitrarily well (Hornik et al., 1989). Training minimizes a loss function with gradient-based optimizers; we use Adam (Kingma and Ba, 2015), whose learning rate plays the role of a step size. Automatic differentiation, as implemented in PyTorch (Paszke et al., 2019), computes exact derivatives of the network output with respect to its input and parameters by applying the chain rule to the recorded computation, without finite-difference approximation.

2.5 Physics-Informed Neural Networks

A PINN (Raissi et al., 2019) represents the unknown solution by a network $N(t;w)$ and trains it on a composite loss. For an inverse problem, unknown model parameters are added to the trainable variables. For system (1) with observations of I, the loss is

$$\mathscr{L}(w,\beta,\gamma) = \mathscr{L}_{\text{data}} + w_{\text{ode}}\,\mathscr{L}_{\text{ode}} + \mathscr{L}_{\text{ic}}, \tag{6}$$

where $\mathscr{L}_{\text{data}}$ is the mean squared error between the network's $I(t)$ and the data, $\mathscr{L}_{\text{ode}}$ is the mean squared residual of the three equations in (1) evaluated by automatic differentiation at a set of collocation points, and $\mathscr{L}_{\text{ic}}$ penalizes deviation from the initial condition. The weight w_{ode} balances fitting the data against satisfying the physics.

2.6 Bootstrap Error Estimation

For real data the true parameters are unknown, so uncertainty is estimated by a residual bootstrap: residuals of the fitted model are resampled with replacement and added back to the fitted values to create synthetic datasets, which are refitted. The 2.5th and 97.5th percentiles of the refitted parameters form an approximate 95% confidence interval.

3 Methods and Applications

3.1 Model and Method Description

Problem setup

Variables: S, I, R (population fractions) as functions of time t in days. Parameters to estimate: $\beta > 0$ and $\gamma > 0$ (per day). Derived quantity: $R_0 = \beta/\gamma$. Assumptions: closed population over one season, homogeneous mixing, constant rates within a season, and no explicit mosquito compartment. Constraints: β, γ positive (enforced by bounds in least squares and by a logarithmic parameterization, $\beta = \exp(\log\beta)$, in the PINN). For the real data, the effective infectious

period $1/\gamma$ is further restricted to 7–60 days, i.e. $\gamma \in [1/60, 1/7]$ per day. Without this constraint, the four-parameter fit to eight monthly points can collapse to a degenerate solution (γ at an arbitrary bound, $R_0 \approx 1$ and a very large N_{eff}) that fits the data about as well. The lower limit reflects the human infectious period, during which the virus circulates in the blood for about 2 to 10 days (Gubler, 1998); the upper limit allows for the mosquito stage, including an extrinsic incubation period of 8 to 12 days (Gubler, 1998), that a plain SIR model absorbs into γ.

Synthetic data

A reference outbreak is generated with $\beta = 0.3$, $\gamma = 0.1$ ($R_0 = 3$, infectious period 10 days), $I_0 = 0.01$, over 100 days, solved by RK4 with $h = 0.1$. Daily values of I are observed with multiplicative Gaussian noise, $I_{\text{obs}} = I_{\text{true}}(1 + \sigma\varepsilon)$ with $\varepsilon \sim N(0,1)$ and $\sigma \in \{0, 0.05, 0.10\}$. Because the true parameters are known, estimation error can be measured exactly.

Real data

Monthly reported dengue cases for the Province of Bulacan were extracted from the OpenDengue V1.3 spatial extract (Clarke et al., 2024), which provides 200 monthly province-level counts from 1994 to 2010. We selected the 6 seasons whose monthly peak exceeded 300 cases (1998, 2001, 2005, 2006, 2008, 2010) and fitted each from May to December (eight months), the period containing the rainy-season wave. Surveillance counts new reported cases rather than the infected fraction, so the observation model is

$$\text{cases in month } k = N_{\text{eff}} \left[S(t_{k-1}) - S(t_k) \right], \tag{7}$$

where $t_k = 30.4k$ days is the end of month k, since every decrease in S is a new infection. The effective population N_{eff} absorbs the reporting rate and the fraction of the population actually susceptible; it is estimated together with β, γ and I_0. This makes N_{eff} much smaller than the province's census population, which is expected when most infections are unreported or occur in a partly immune population.

3.2 Implementation and Code

All code is written in Python using NumPy, SciPy, pandas, Matplotlib and PyTorch, and is available in the project repository (`https://github.com/SeanXxotick/MCS306FCO`). The settings (seeds, step sizes, tolerances, epochs, loss weights) are in `src/config.py`, except the parameter bounds and starting points of the fits, which are in `src/classical.py`; `run_all.py` regenerates every table and figure in this paper.

Algorithm 1: RK4 solver

The RK4 step (2)–(3) is implemented directly (`src/sir.py`):

```
def rk4(beta, gamma, y0, t_end, h):
    n = int(round(t_end / h))
    y = np.asarray(y0, dtype=float)
    for k in range(n):
        k1 = sir_rhs(y, beta, gamma)
        k2 = sir_rhs(y + 0.5 * h * k1, beta, gamma)
        k3 = sir_rhs(y + 0.5 * h * k2, beta, gamma)
        k4 = sir_rhs(y + h * k3, beta, gamma)
        y = y + h / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)
```

Algorithm 2: Classical solve-then-fit

For a candidate $\theta = (\beta, \gamma)$, the residual vector is computed by solving (1) with RK4 and comparing the model's $I(t_i)$ with the data; `least_squares` then updates θ (`src/classical.py`). For the real data, $\theta = (\beta, \gamma, \log_{10} N_{\text{eff}}, \log_{10} I_0)$ and the residuals follow (7). Because this four-parameter problem is non-convex, the fit is started from 6 initial guesses and the best solution is kept. When h does not divide the one-day spacing of the synthetic observations ($h = 2$ and 5 days in Experiment 1b), the model values at the observation times are linearly interpolated between RK4 points. The synthetic fits use the tolerances in Table 1; the real-data fits and the bootstrap refits use SciPy's default of 10^{-8}.

```
def residuals(p):
    t, Y = rk4(p[0], p[1], y0, t_end, h)
    I_model = Y[::stride, 1] if stride else np.interp(t_obs, t, Y[:, 1])
    return (I_model - I_obs) / I_obs.max()

res = least_squares(residuals, p0, bounds=([1e-4, 1e-4], [5, 5]),
                    ftol=1e-10, xtol=1e-10, gtol=1e-10)
```

Algorithm 3: PINN

The network maps rescaled time $\tau = t/T \in [0,1]$ to $(S, I/s_I, R)$ through 3 hidden layers of 32 tanh units and a sigmoid output, where s_I is the maximum observed I (synthetic data), so all outputs are of order one. The ODE residuals are evaluated at 200 equally spaced collocation points using `torch.autograd.grad`, with the chain rule $d/d\tau = T\, d/dt$. β and γ are stored as log-values and optimized by Adam together with the weights (`src/pinn.py`):

```
out = net(tau_c); S, I_s, R = out[:, :1], out[:, 1:2], out[:, 2:3]
I = I_s * s_I
r_S = d_dtau(S) - T * (-beta * S * I)
r_I = d_dtau(I_s) - T * (beta * S * I - gamma * I) / s_I
r_R = d_dtau(R) - T * (gamma * I)
loss = l_data + w_ode * (r_S**2 + r_I**2 + r_R**2).mean() + l_ic
loss.backward(); opt.step()
```

For the real data the PINN is trained on monthly case counts instead (`train_pinn_incidence`): the data loss compares $N_{\text{eff}}[S(t_{k-1}) - S(t_k)]$ at the month ends with the reported cases, both divided by the largest monthly count; N_{eff} and I_0 are fixed at the classical estimates; s_I is fixed at 0.2; the initial-condition term is weighted by 100 because I_0 is very small; and training runs for a fixed 20,000 epochs without the stopping rule.

Step size, iteration count and tolerance

Table 1 lists the numerical settings. The global error of RK4 is $|S_h(T) - S_{\text{ref}}(T)| + |I_h(T) - I_{\text{ref}}(T)|$ at $T = 100$ days, where the reference solution uses $h = 0.001$. Estimation errors are reported as relative parameter errors $|\hat{\theta} - \theta|/\theta \times 100\%$, and as the root-mean-square error (trajectory RMSE) between the true $I(t)$ and an RK4 re-simulation with the estimated parameters. Runtimes were measured on a laptop CPU (Intel Core i5-13420H) with PyTorch restricted to one thread, running the four experiments as parallel processes; environment details are recorded in `results/run_log.txt`.

Table 1. Numerical settings.

Component	Setting	Value
RK4 (data generation)	step size h	0.1 day
RK4 (convergence study)	step sizes h	4, 2, 1, 0.5, 0.25, 0.125 (reference $h = 0.001$)
RK4 (inside synthetic fit)	step size h	0.1 day (Exp. 1b varies 5, 2, 1, 0.5, 0.1)
RK4 (real-data model)	step size h	0.5 day
Least squares (synthetic)	tolerances ftol, xtol, gtol	10^{-10}
Least squares (real data)	tolerances ftol, xtol, gtol	10^{-8} (SciPy default)
PINN	Adam learning rate	0.001
PINN	maximum epochs	20,000
PINN	stopping rule	relative change of β and γ over 500 epochs < 0.0001
PINN	physics-loss weight w_{ode}	0.01 (Exp. 3 varies 1, 0.1, 0.01, 0.001)
PINN (real data)	epochs (fixed)	20,000
PINN (real data)	initial-condition weight; s_I	100; 0.2
Bootstrap	replicates	50

3.3 Results and Visualization

Experiment 1: RK4 convergence

Table 2 and Figure 1a show that halving h reduces the error by a factor of about 16; the observed order ranges from 3.984 to 4.005, in agreement with the theoretical order of four. Population is conserved to machine precision at every step size. Figure 1b shows that the solver step used inside the fit controls a systematic bias: with $h = 5$ days the estimates of β and γ are off by 0.36% and 0.42%, while with $h = 1$ day the error of β drops to $3.4 \times 10^{-4}\%$. For $h = 2$ and

Table 2. Global error of the RK4 solver against a reference solution ($h = 0.001$).

Step size h	**Steps**	**Global error at** $t = 100$	**Observed order** p	$\max \lvert S+I+R-1 \rvert$
4	25	1.698×10^{-5}	–	2.2×10^{-16}
2	50	1.057×10^{-6}	4.005	2.2×10^{-16}
1	100	6.681×10^{-8}	3.984	4.4×10^{-16}
0.5	200	4.212×10^{-9}	3.987	8.9×10^{-16}
0.25	400	2.646×10^{-10}	3.993	8.9×10^{-16}
0.125	800	1.658×10^{-11}	3.996	1.3×10^{-15}

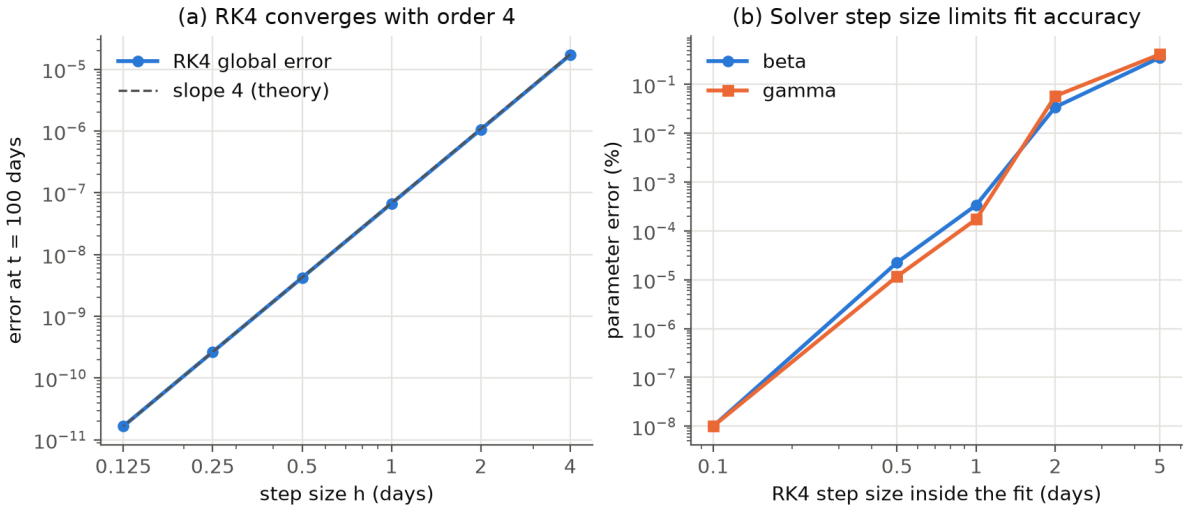


Figure 1. (a) Log–log plot of RK4 global error versus step size with a slope-4 reference line. (b) Relative error of the classical estimates on noise-free data as a function of the RK4 step size used inside the fit; errors below $10^{-8}\%$ are drawn at $10^{-8}\%$.

5 days the model is also linearly interpolated between RK4 points (Algorithm 2), so part of this bias comes from the interpolation rather than from RK4 itself. With $h = 0.1$, the same step used to generate the data, the parameters are recovered to machine precision; this is the so-called inverse crime, which is why noise is added in the remaining experiments.

Experiment 2: Parameter recovery on synthetic data

Table 3. Recovery of $\beta = 0.3$ and $\gamma = 0.1$ from synthetic data.

Method	Noise	$\hat{\beta}$ (rel. error)	$\hat{\gamma}$ (rel. error)	Traj. RMSE	Iter.	Time (s)
Classical	0%	0.3000 (3.7×10^{-14}%)	0.1000 (1.4×10^{-14}%)	1.39×10^{-16}	6	0.55
Classical	5%	0.2998 (0.27%)	0.1000 (0.40%)	9.97×10^{-4}	6	0.46
Classical	10%	0.2995 (0.53%)	0.1000 (0.81%)	1.99×10^{-3}	7	0.51
PINN	0%	0.2994 (0.21%)	0.1000 (0.03%)	5.59×10^{-4}	13,000	72.91
PINN	5%	0.3000 (2.8×10^{-3}%)	0.0994 (0.63%)	1.03×10^{-3}	10,500	58.38
PINN	10%	0.3002 (0.06%)	0.0987 (1.25%)	2.11×10^{-3}	8,500	48.49

Note. Classical values are means over 5 noise draws (one draw at 0% noise); iterations are SciPy's function evaluations, each a full RK4 solve (the finite-difference Jacobian adds two more solves per iteration). PINN values are from one training run (seed 0); iterations are epochs until the stopping rule.

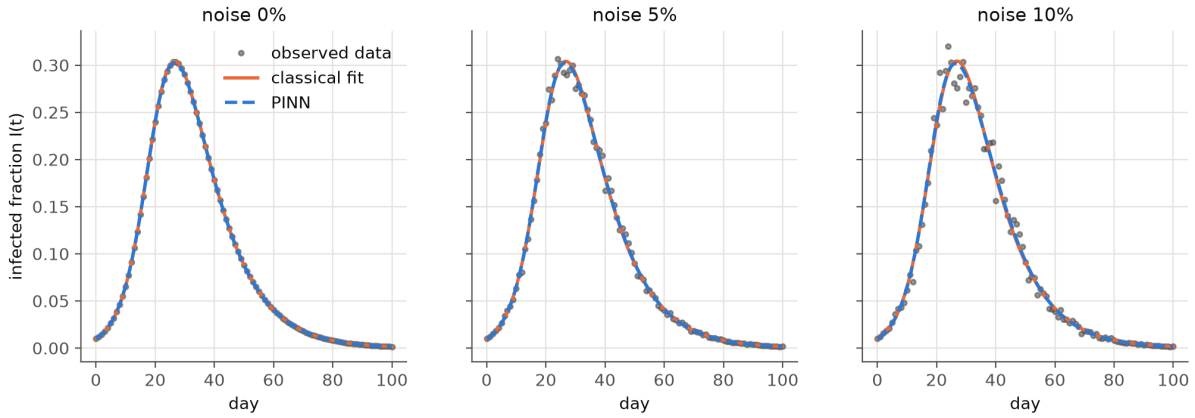


Figure 2. Observed infected fraction (points) with the classical fit (solid) and PINN prediction (dashed) at 0%, 5% and 10% noise.

Both methods recover the parameters accurately (Table 3, Figure 2). At 10% noise the PINN's errors are 0.06% for β and 1.25% for γ, versus 0.53% and 0.81% on average for the classical method. The classical method converges in at most 7 function evaluations (about 0.51 s), whereas the PINN needs thousands of epochs (on average 60 s), roughly 118 times longer. Figure 3 shows the PINN's convergence behavior: the data loss falls quickly, the ODE residual is reduced more slowly, and β and γ settle onto the true values before the stopping rule ends training.

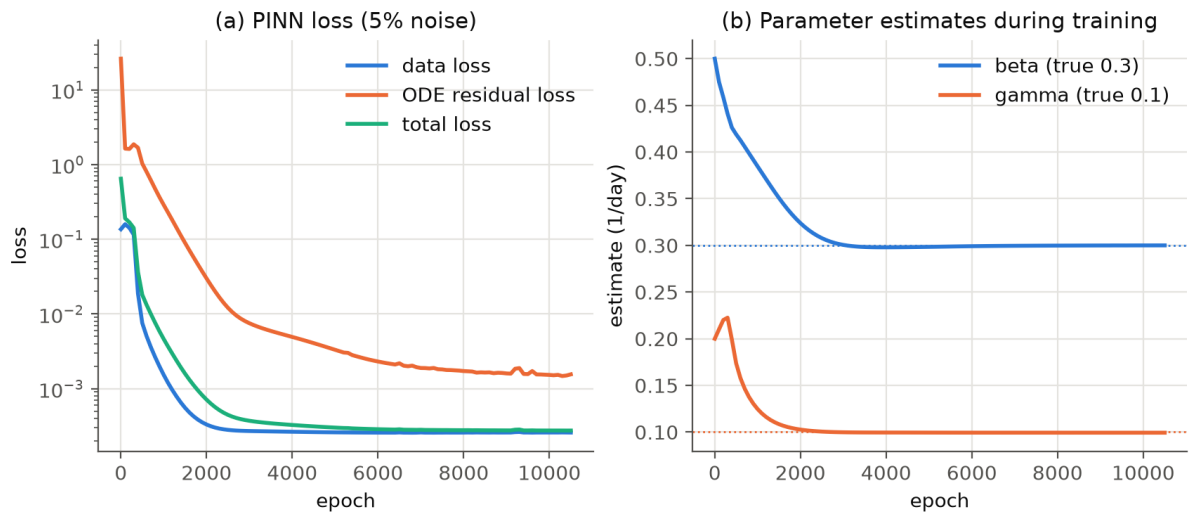


Figure 3. PINN training at 5% noise. (a) Data loss, ODE-residual loss (before weighting) and total loss, recorded every 100 epochs. (b) Estimates of β and γ during training; dotted lines mark the true values.

Table 4. PINN estimates at 5% noise for different physics-loss weights.

w_{ode}	$\hat{\beta}$	β error	$\hat{\gamma}$	γ error	Epochs	Time (s)
1	0.2874	4.19%	0.0989	1.10%	20,000	112.8
0.1	0.2972	0.94%	0.0993	0.71%	11,500	65.5
0.01	0.3000	2.8×10^{-3}%	0.0994	0.63%	10,500	56.4
0.001	0.3049	1.63%	0.0995	0.53%	12,000	55.6

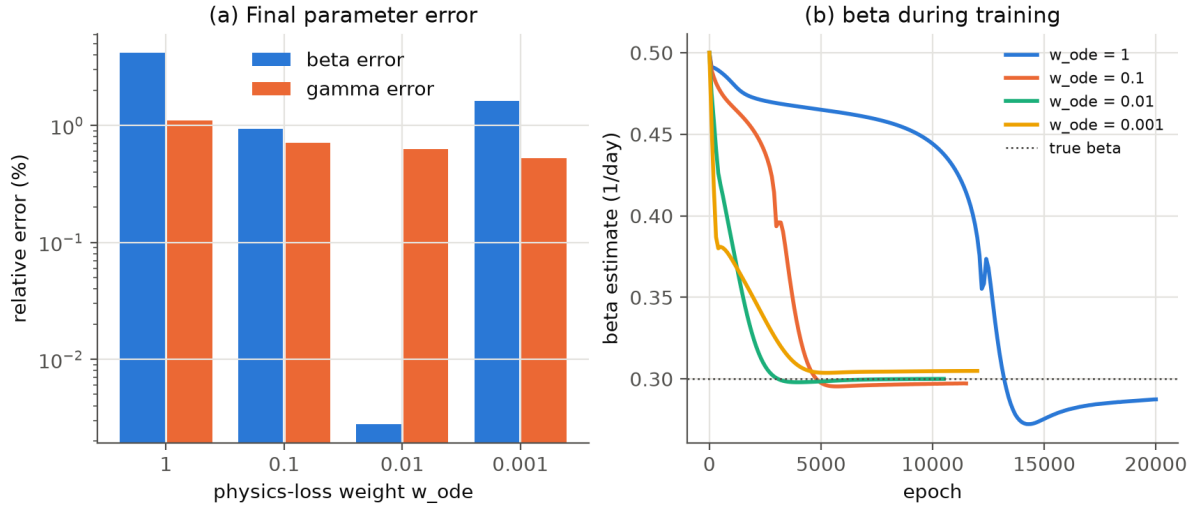


Figure 4. (a) Final relative error of β and γ (log scale) for each physics-loss weight. (b) The estimate of β during training for each weight.

Experiment 3: Sensitivity to the physics-loss weight

The physics-loss weight changes both the accuracy and the convergence of the PINN (Table 4, Figure 4). The most accurate weight was $w_{\text{ode}} = 0.01$ (errors 2.8×10^{-3}% and 0.63%), while $w_{\text{ode}} = 1$ gave errors of 4.19% and 1.10%. With the largest weight ($w_{\text{ode}} = 1$) the estimate of β stayed far from the true value for more than half of training (Figure 4b) before moving, and the run reached the 20,000-epoch cap without satisfying the stopping rule. With the smallest weight ($w_{\text{ode}} = 0.001$) the β error rose to 1.63% (γ error 0.53%) after 12,000 epochs. The classical method has no corresponding tuning parameter.

Experiment 4: Dengue seasons in Bulacan

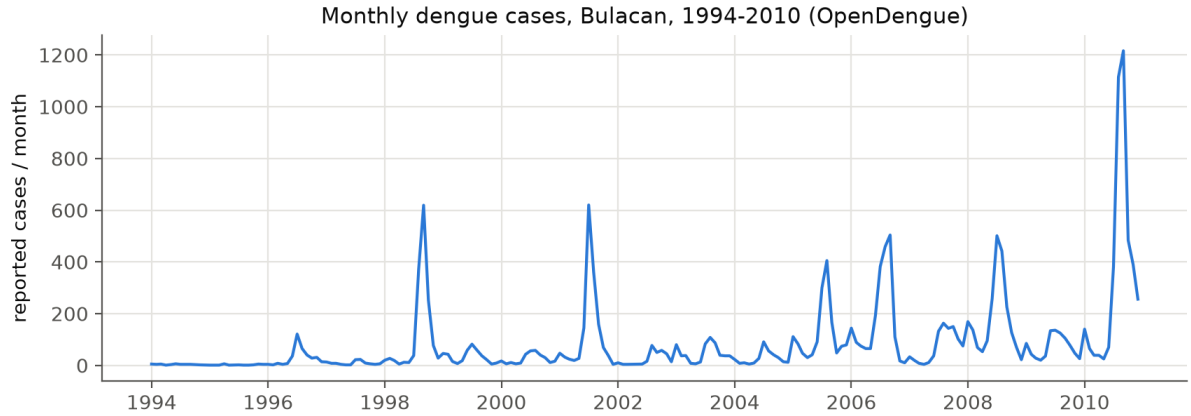


Figure 5. Monthly reported dengue cases in Bulacan, 1994–2010 (OpenDengue V1.3). The line bridges the 4 months with no reported count.

Table 5. Classical SIR fits to Bulacan outbreak seasons (May–December).

Season	Cases (May–Dec)	Peak (cases)	$\hat{\beta}$	$\hat{\gamma}$	R_0	$1/\hat{\gamma}$ (d)	N_{eff}	R^2
1998	1,408	619	0.120	0.045	2.64	22.1	1,544	0.998
2001	1,423	620	0.121	0.043	2.79	23.2	1,449	0.976
2005	1,204	405	0.082	0.024	3.43	41.8	1,148	0.930
2006	1,740	504	0.184	0.143	1.29	7.0	4,397	0.903
2008	1,739	501	0.072	0.027	2.62	36.5	1,944	0.994
2010	3,936	1,216	0.083	0.030	2.72	32.8	4,220	0.952

Note. $1/\hat{\gamma}$ was constrained to 7–60 days; the 2006 estimate lies on the boundary.

The classical SIR model reproduces the shape of each season (Table 5, Figure 6), with R^2 from 0.903 to 0.998 and R_0 estimates from 1.29 to 3.43. For 2010, the largest season (3,936 cases from May to December), the fit gives $\beta = 0.083$ per day, $\gamma = 0.030$ per day and $R_0 = 2.72$. The bootstrap intervals in Table 6 quantify how well eight monthly points constrain the parameters: R_0 lies between 1.29 and 4.24. On the same season the PINN's network output matches the data ($R^2 = 0.9997$), but solving the SIR model with the PINN's β and γ gives $R^2 = 0.647$ (Table 7, Figure 7), meaning the learned parameters are not consistent with the network's own curve.

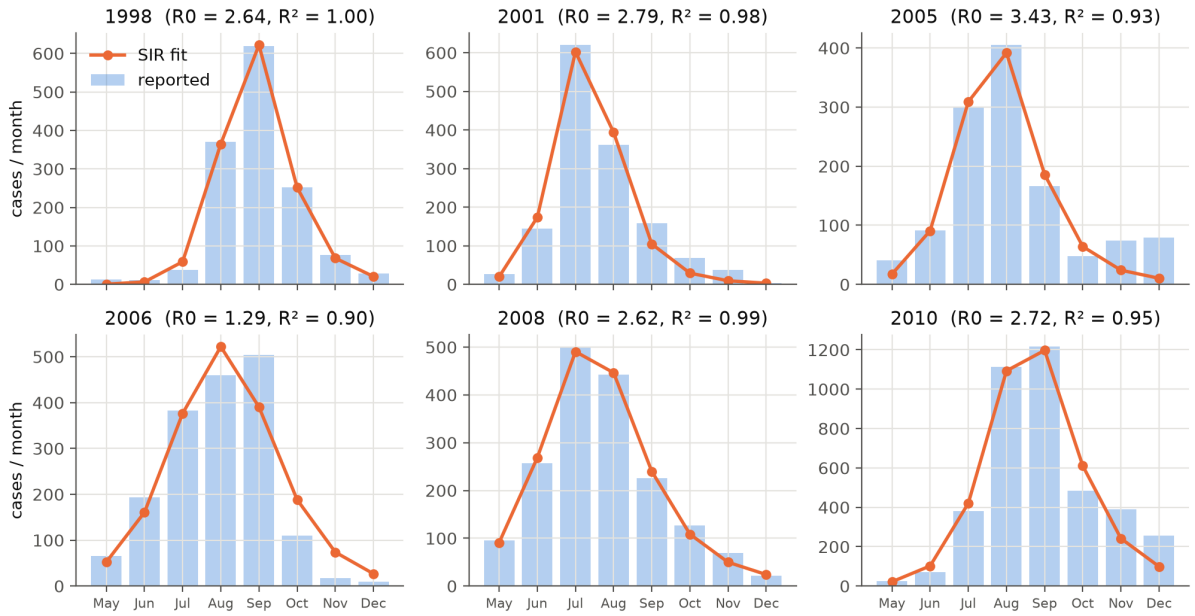


Figure 6. Reported monthly cases (bars) and classical SIR fits (lines) for six outbreak seasons.

Table 6. Residual-bootstrap 95% intervals for the 2010 season (50 replicates).

Parameter	**Estimate**	**95% CI (lower)**	**95% CI (upper)**
β	0.083	0.063	0.187
γ	0.030	0.017	0.143
R_0	2.715	1.292	4.236
N_{eff}	4,220	3,823	9,683

Table 7. Classical fit and PINN on the 2010 season.

Method	$\hat{\beta}$	$\hat{\gamma}$	R_0	R^2 **(own curve)**	R^2 **(RK4 re-sim.)**	**Time (s)**
Classical (RK4 + least squares)	0.083	0.030	2.72	0.952	0.952	12.9
PINN (N_{eff}, I_0 fixed)	0.104	0.045	2.32	1.000	0.647	107.9

Note. For the PINN, N_{eff} and I_0 were fixed at the classical estimates so that only β and γ were learned. "Own curve" is the network output; "RK4 re-simulation" solves the SIR model with the PINN's β and γ.

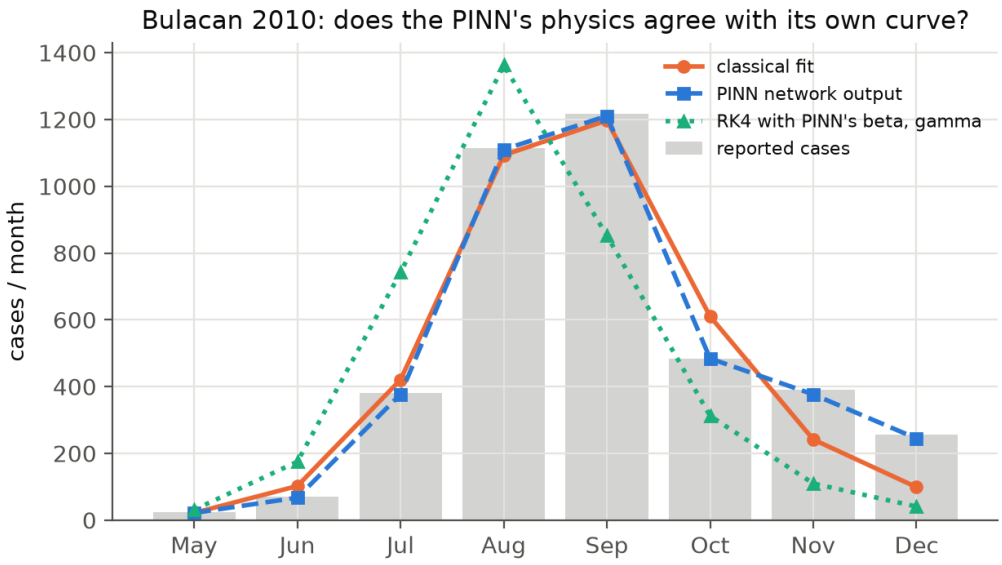


Figure 7. Bulacan 2010: reported cases, classical fit, the PINN network's output, and an RK4 re-simulation using the PINN's estimated β and γ.

3.4 Analysis and Discussion

Accuracy. On synthetic data both methods achieve parameter errors of at most 1.25% (Table 3). The classical errors, averaged over several noise draws, grow with the noise level as expected; the PINN errors come from a single training run per noise level, so a lower PINN error in one run should not be read as a systematic advantage. Since both methods fit the same model to the same data, neither can do much better than the information in the noisy observations allows. The classical method's accuracy is limited by the RK4 step used inside the fit (Experiment 1b), which is negligible for $h \leq 1$ day because of the fourth-order convergence.

Stability. RK4 is stable for the step sizes used, and least squares converged in every synthetic run from the same initial guess used by the PINN. The PINN is less robust: its result depends on the physics-loss weight (Experiment 3). When the physics term dominates, the network first learns a smooth curve that satisfies the ODE for the current, incorrect parameters, and β and γ then move only slowly; when it is too weak, the data alone determine the curve and the parameters are less constrained. This agrees with known training pathologies of PINNs, where imbalanced loss gradients slow or misdirect learning (Wang et al., 2021).

Computational cost. One RK4 solve costs $O(T/h)$ evaluations of $\mathbf{f}$, and the classical fit needs only tens of solves. The PINN's cost per epoch is a forward and backward pass over the collocation and data points, repeated for thousands of epochs. In our experiments the PINN was roughly 118 times slower. For a two-parameter ODE this cost is not justified; PINNs become attractive for partial differential equations, high-dimensional problems, or when no efficient solver exists.

Real data. The classical SIR model captures the timing and size of Bulacan's outbreaks, but the implied infectious periods ($1/\gamma \approx 22$–42 days for the seasons whose estimate lies inside the constraint) are longer than the 2 to 10 days during which the virus circulates in a patient's blood

(Gubler, 1998). This is a symptom of model simplification: SIR has no mosquito compartment, so the extrinsic incubation period in the mosquito (8 to 12 days) and the mosquito lifespan are absorbed into γ. β and γ should therefore be read as effective seasonal rates rather than biological constants. The 2006 season ended at the boundary of the constraint, which signals that the data are not well described by SIR there: in 2006 reported cases fell abruptly from 504 in September to 110 in October and 17 in November, a drop faster than the SIR curve can follow with a plausible infectious period, possibly driven by the end of the rainy season or by changes in reporting. Monthly data also provide only eight observations for four unknowns, and the bootstrap intervals show the resulting uncertainty. The PINN struggled most here: with eight points, the network can interpolate the data without the ODE residual being small enough to pin down β and γ.

Practical significance and limitations. For provincial health offices working with sparse monthly surveillance data, a classical SIR fit is fast, transparent and adequate for describing an outbreak's growth (R_0) and timing. Our conclusions are limited by: the SIR simplification (no vector, no seasonality in β, no under-reporting model beyond N_{eff}); data that end in 2010 and are aggregated monthly; one PINN architecture and one training seed per configuration; and treating each season independently.

4 Conclusion

We compared a physics-informed neural network with a classical RK4 plus least-squares method for estimating SIR transmission parameters. The RK4 solver was verified to converge with fourth-order accuracy. On synthetic outbreaks with up to 10% noise, both methods recovered β and γ within a few percent, but the PINN was about 118 times more expensive and sensitive to its physics-loss weight. On real dengue data from Bulacan, the classical model fitted 6 outbreak seasons with $R^2 \geq 0.90$ and effective R_0 between 1.29 and 3.43, while the PINN failed to produce parameters consistent with its own fit from eight monthly observations. For low-dimensional epidemic models with sparse data, classical fitting is the more effective method; PINNs are better suited to richer data or more complex models.

Future work suggested by these findings, and a natural basis for a thesis, includes:

- replacing SIR with a host–vector dengue model (Esteva and Vargas, 1998) so that the estimated rates become biologically interpretable;
- using weekly or more recent surveillance data (e.g., from the Department of Health) to give the PINN more observations;
- letting the transmission rate vary with rainfall and temperature, a setting where a neural network can represent an unknown function $\beta(t)$ that a classical fit cannot easily handle;
- fitting several seasons jointly with shared parameters; and
- improving PINN training with adaptive loss weighting or two-stage training that fixes the network before estimating β and γ.

References

Bhatt, S., Gething, P. W., Brady, O. J., Messina, J. P., Farlow, A. W., Moyes, C. L., Drake, J. M., Brownstein, J. S., Hoen, A. G., Sankoh, O., Myers, M. F., George, D. B., Jaenisch, T., Wint, G. R. W., Simmons, C. P., Scott, T. W., Farrar, J. J., & Hay, S. I. (2013). The global distribution and burden of dengue. *Nature*, 496(7446), 504–507. https://doi.org/10.1038/nature12060

Burden, R. L., & Faires, J. D. (2011). *Numerical analysis* (9th ed.). Brooks/Cole, Cengage Learning.

Clarke, J., Lim, A., Gupte, P., Pigott, D. M., van Panhuis, W. G., & Brady, O. J. (2024). A global dataset of publicly available dengue case count data. *Scientific Data*, 11, Article 296. https://doi.org/10.1038/s41597-024-03120-7

Esteva, L., & Vargas, C. (1998). Analysis of a dengue disease transmission model. *Mathematical Biosciences*, 150(2), 131–151. https://doi.org/10.1016/S0025-5564(98)10003-2

Gubler, D. J. (1998). Dengue and dengue hemorrhagic fever. *Clinical Microbiology Reviews*, 11(3), 480–496. https://doi.org/10.1128/CMR.11.3.480

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. https://doi.org/10.1016/0893-6080(89)90020-8

Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3(6), 422–440. https://doi.org/10.1038/s42254-021-00314-5

Kermack, W. O., & McKendrick, A. G. (1927). A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772), 700–721. https://doi.org/10.1098/rspa.1927.0118

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024–8035). Curran Associates.

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5), A3055–A3081.

https://doi.org/10.1137/20M1318043